EXPERIMENT - 59

1. Problem Statement

Write a C program that reads a string s. Pass the string to a function named longest_unique_substring() using its address. Use pointers to pass the string between the calling function and longest_unique_substring(). The function longest_unique_substring() must:

- Find the longest substring of s that contains no repeated characters.
- Calculate and display the length of the longest substring.
- Treat uppercase and lowercase letters as different characters.
- Display 0 if the input string is empty.

For abcabcbb the answer is 3, from abc. For bbbbb it is 1, from b. For pwwkew it is 3, from wke. For an empty string the output is 0.

2. Algorithm

Main Program

Step 1: START

Step 2: Read the string s using fgets().

Step 3: Replace the '\n' at the end of s with '\0'.

Step 4: Call longest_unique_substring(s).

Step 5: STOP

Function longest_unique_substring()

Step 1: If s[0] is '\0', display 0 and return to the calling function.

Step 2: Set max = 0 and repeat for i = 0 while s[i] is not '\0':

Set len = 0.

Repeat for j = i while s[j] is not '\0':

Set repeated = 0, and for k = i to j − 1, if s[k] is equal to s[j] set repeated = 1 and leave the k loop.

If repeated is 1, leave the j loop, otherwise add 1 to len.

If len is greater than max, set max = len.

Step 3: Display max.

Step 4: Return to the calling function.

3. Test Case Table

#	String	Expected Output	Actual Output	Result
1	abcabcbb	3, from abc	Matches example 1 in the question	**PASS**
2	bbbbb	1, as only a single b can be taken	Matches example 2	**PASS**

#	String	Expected Output	Actual Output	Result
3	pwwkew	3, from wke and not pw	Matches example 3	**PASS**
4	empty line	0	Matches example 4	**PASS**
5	abcdef	6, since nothing repeats at all	6, the whole string	**PASS**
6	aAbB	4, as a and A count as different characters	4, so the check is case sensitive	**PASS**
7	dvdf	3, from vdf	3, not 2	**PASS**
8	abba	2, from ab or ba	2	**PASS**

4. Source Code

longestunique59.c

```
/* Experiment 42. One function for each of the three operations.
   Every option reads its own input here, which is different from what
   I have been doing. Option 3 needs two strings and the other two need
   one, so there was no single string I could read at the start that
   would have suited all three.
   The question lists only three operations, but a do while menu has no
   way of stopping, so option 4 is mine. Without it the menu keeps
   coming back and the only way out is to kill the program. */

#include <stdio.h>

void removeVowels(const char s[], char result[]);
char firstRepeatingLowercase(const char s[]);
int areAnagrams(const char s1[], const char s2[]);

int main(void)
{
    char str[100], str2[100], result[100];
    char ch;
    int choice;

    do {
        printf("\nMENU\n");
        printf("1. Remove all the vowels from a string\n");
        printf("2. Find the first repeating lowercase letter\n");
        printf("3. Check whether two strings are anagrams\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                removeVowels(str, result);
                printf("String without vowels: %s\n", result);
                break;

            case 2:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                ch = firstRepeatingLowercase(str);
                if (ch == '\0')
                    printf("No lowercase letter repeats in this string\n");
                else
                    printf("First repeating lowercase letter: %c\n", ch);
                break;

            case 3:
                printf("Enter the first string: ");
                scanf(" %[^\n]", str);
                printf("Enter the second string: ");
                scanf(" %[^\n]", str2);
                if (areAnagrams(str, str2))
                    printf("The two strings are anagrams\n");
                else
                    printf("The two strings are not anagrams\n");
                break;

            case 4:
                printf("Exiting the program.\n");
                break;
        }
    } while (choice != 4);

    return 0;
}
```

```
/* i walks through the old string and j marks where to write in the new
   one, and they drift apart because j only moves when something is
   actually copied.
   The '\0' on the last line is the one I forgot. Programming in C came
   out right and then had junk stuck on the end of it, because printf
   had no idea where the new string finished.
   Both cases are listed because A and a are different characters, and
   leaving the capitals out would have let the A through in Day. */
void removeVowels(const char s[], char result[])
{
    int i, j = 0;
    char ch;

    for (i = 0; s[i] != '\0'; i++) {
        ch = s[i];

        if (ch != 'a' && ch != 'e' && ch != 'i' && ch != 'o' && ch != 'u' &&
            ch != 'A' && ch != 'E' && ch != 'I' && ch != 'O' && ch != 'U') {
            result[j] = ch;
            j++;
        }
    }

    result[j] = '\0';
}

/* First repeating can be read two ways, and the two give different
   answers. In abcdbca one reading says b, the first letter you meet
   that you have seen before. The other says a, because out of the
   letters that repeat at all, a is the one that showed up earliest.
   I went with the second, which is the usual meaning, and that is why
   the counting happens first and the walk from the left happens after.
   One single pass would have given the other answer.
   count has 26 boxes, one per small letter, and s[i] - 'a' turns a
   letter straight into its box number, since 'a' is 97 and 'b' is 98.
   Capitals are skipped in both loops, so AABBcdc answers c, even though
   A and B repeat more times than c does. */
char firstRepeatingLowercase(const char s[])
{
    int count[26] = {0};
    int i;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z')
            count[s[i] - 'a']++;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z' && count[s[i] - 'a'] > 1)
            return s[i];

    /* '\0' is not a letter, so main can use it to mean nothing was
       found. */
    return '\0';
}

/* Anagrams hold the same characters the same number of times, only in
   a different order. My first idea was to sort both strings and see if
   they came out identical, but counting turned out to be less code, and
   it leaves the two strings alone instead of rearranging them.
   256 boxes covers every character there is, so spaces and digits are
   counted along with the letters, and nothing is quietly ignored.
   The length test is not only a shortcut. The counts could agree on
   every character and the strings still be different lengths if one of
   them held something the other did not. */
int areAnagrams(const char s1[], const char s2[])
{
    int count1[256] = {0};
    int count2[256] = {0};
    int i, len1 = 0, len2 = 0;

    for (i = 0; s1[i] != '\0'; i++) {
        count1[(unsigned char) s1[i]]++;
        len1++;
```

```
    }

    for (i = 0; s2[i] != '\0'; i++) {
        count2[(unsigned char) s2[i]]++;
        len2++;
    }

    if (len1 != len2)
        return 0;

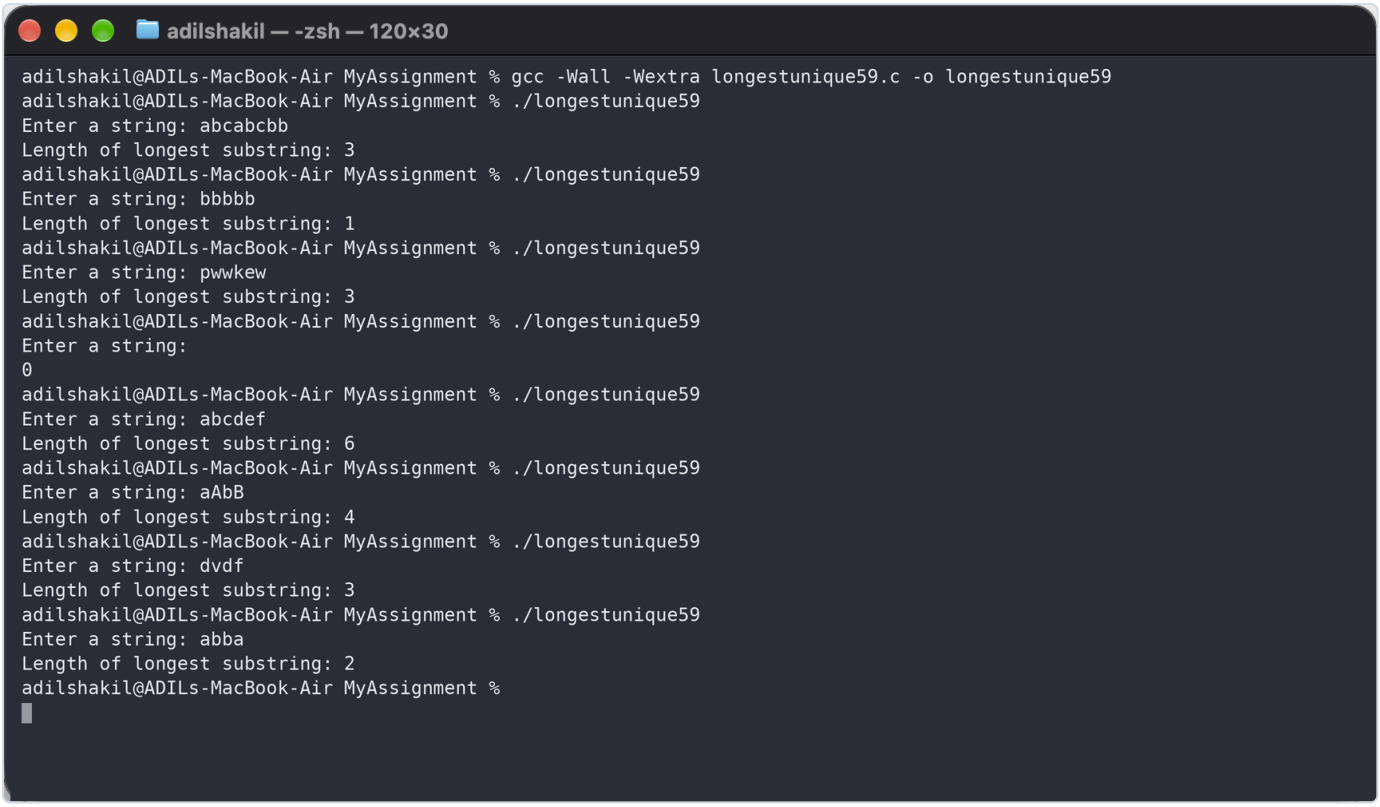
    for (i = 0; i < 256; i++)
        if (count1[i] != count2[i])
            return 0;

    return 1;
}
```

5. Compilation and Output

Compiled with `gcc -Wall -Wextra longestunique59.c -o longestunique59` so the small warnings would show up too. It built clean.

All Eight Runs



All eight cases came out right, and the first four are the examples given in the question.

The reading is done with fgets and not scanf, and that is because of example 4. On a blank line `scanf(" %[^\n]", s)` simply fails and leaves the array untouched, so an empty string could never be read at all. fgets takes the line as it is, even when there is nothing in it. It does keep the Enter key as a '\n' though, which would otherwise be counted as a character, so that is cut off before the string is passed on.

The function itself is three loops. i chooses where the substring starts, j walks forward from there, and k looks back over the part already taken to see whether the new character is one of them. The moment it is, the run ends and i moves on.

Nothing is converted to lower case anywhere, which is what keeps a and A separate. Case 6 is the test for that: aAbB gives 4 and not 2.

Case 7 is the one I would have got wrong with a careless method. In dvdf the answer is vdf, which is 3, and it starts in the middle of the string rather than at the beginning. Because i tries every starting point in turn, that one is found like any other.

Case 5 is the opposite end, a string where nothing repeats, so the answer is the whole length.